

UD.1 Tecnologías de las Aplicaciones Web

**Implantación de Aplicaciones Web (IAW)
CFGS 2º ASIR**

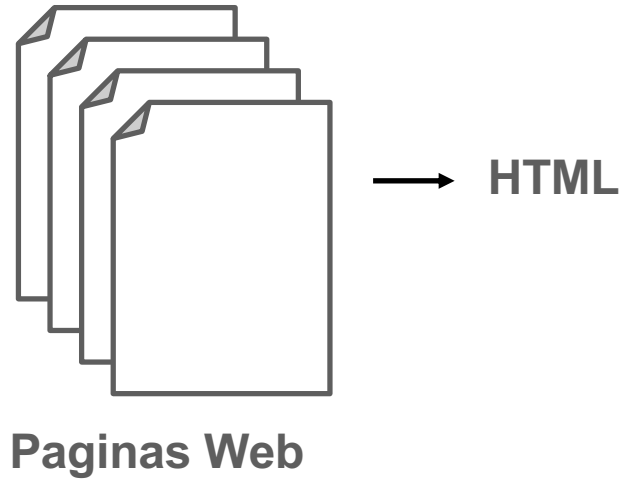
Sergio Pérez



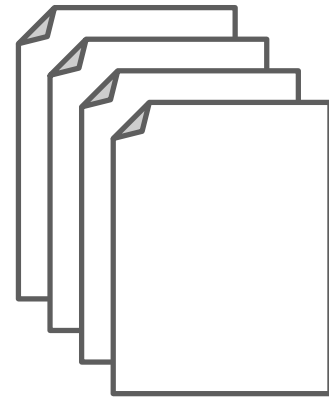
Contenidos de la Unidad

- **Arquitectura de una aplicación Web**
 - Arquitectura Cliente/Servidor
- **Servidores y páginas web**
 - Servidores Web
 - Tipos de Páginas Web
- **Lenguajes de programación en Web**
 - Lenguajes del lado Cliente
 - Lenguajes del lado Servidor
- **Preparación del entorno de desarrollo**
 - Herramientas y entornos de programación
 - Herramientas de gestión y administración de bases de datos
 - Pilas de Software
 - Herramientas para el uso de PHP
 - Herramientas de control de versiones (Git y Github)

Arquitectura de una aplicación Web



Arquitectura de una aplicación Web



Paginas Web

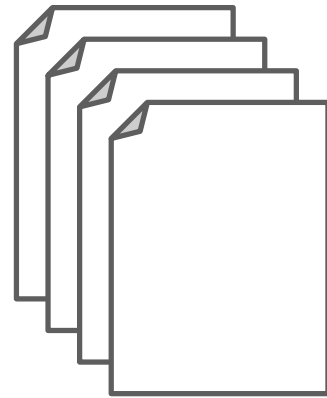


HTML

Protocolos de envío de información:

- HTTP
- HTTPS
- FTP
- SSH

Arquitectura de una aplicación Web



Paginas Web

→ HTML

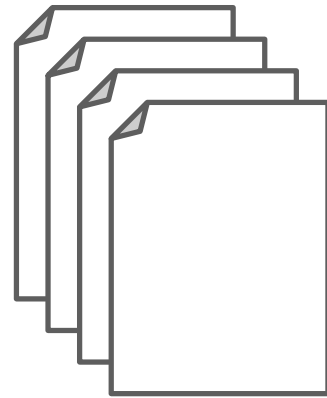
Protocolos de envío de información:

- HTTP
- HTTPS
- FTP
- SSH

 <https://www.elmundotoday.com>



Arquitectura de una aplicación Web



Paginas Web

→ HTML

Protocolos de envío de información:

- HTTP
- HTTPS
- FTP
- SSH

 <https://www.elmundotoday.com>



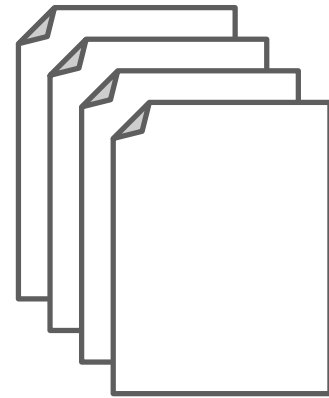
HTTPS

Mándame tu página



**SERVIDOR
ELMUNDOTODAY**

Arquitectura de una aplicación Web



Paginas Web

→ HTML

Protocolos de envío de información:

- HTTP
- HTTPS
- FTP
- SSH

 <https://www.elmundotoday.com>



HTTPS

Mándame tu página



Index.html

Yo te mando este trozo de texto
y tu sabrás que hacer con él



**SERVIDOR
ELMUNDOTODAY**

Arquitectura de una aplicación Web



SERVIDOR WEB

- Almacenan las páginas web para servir las bajo demanda.
- Están disponible 24/7 y 365 días al año.
- Ofrecen diferentes servicios a clientes
- Pueden servir varias peticiones a la vez
- Escuchan en un puerto concreto esperando la llegada de nuevas peticiones

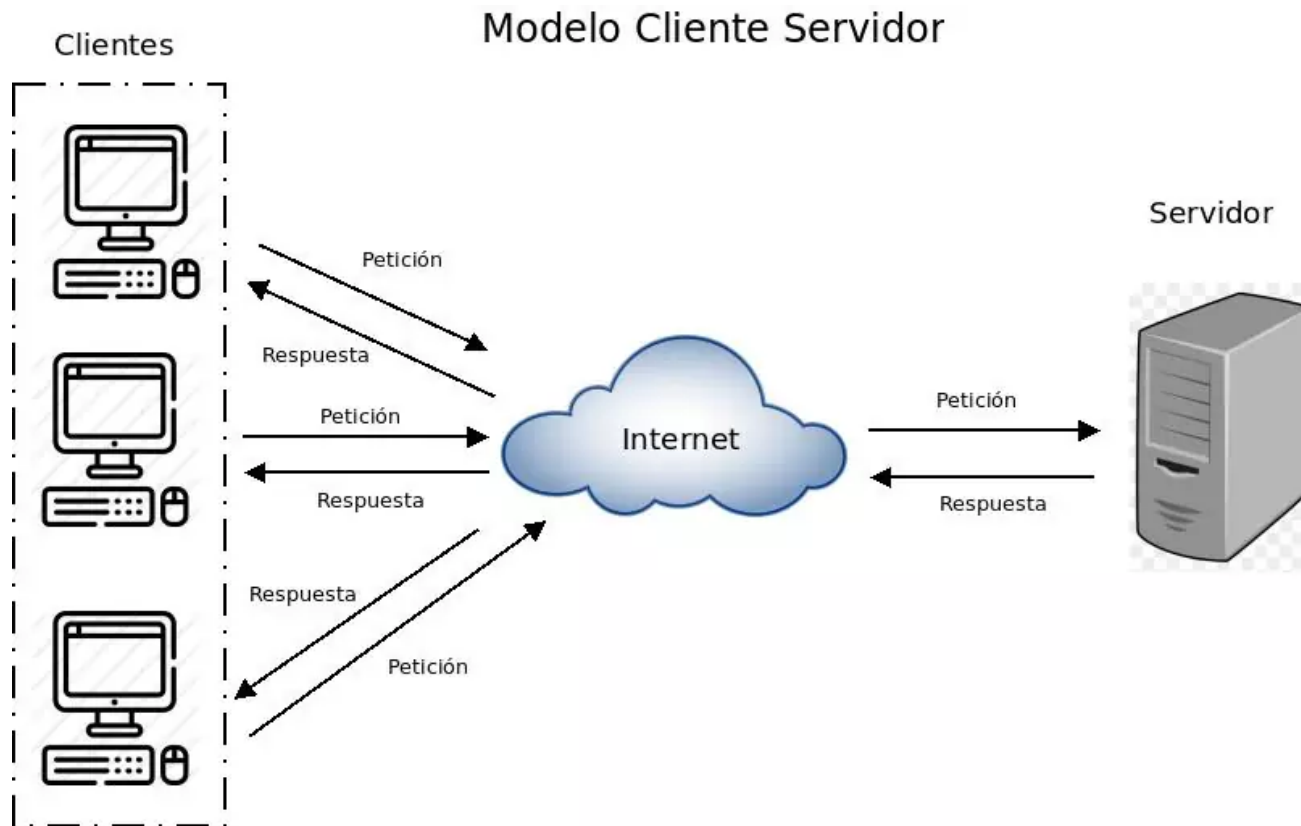
Para acceder a los datos de estos servidores debemos conocer su **protocolo**, su **dominio** y la **ruta del archivo** al que queremos acceder. Toda esta información podemos encontrarla en su URL (*Uniform Resource Locator*)

URL: **Protocolo://Servicio.Dominio/Ruta_completa_del_recurso**

URL: **<https://www.amazon.es/index.html>**

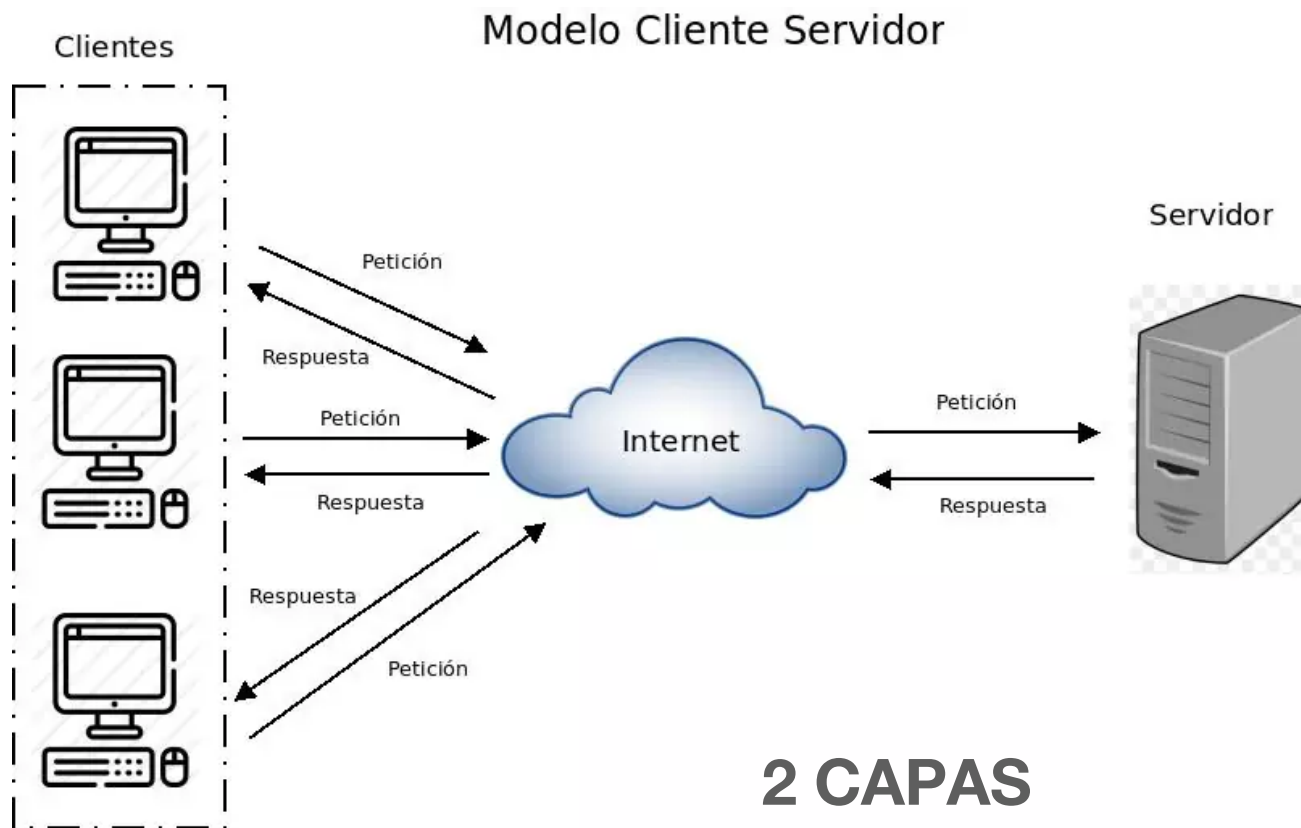
Arquitectura de una aplicación Web

Arquitectura Cliente/Servidor



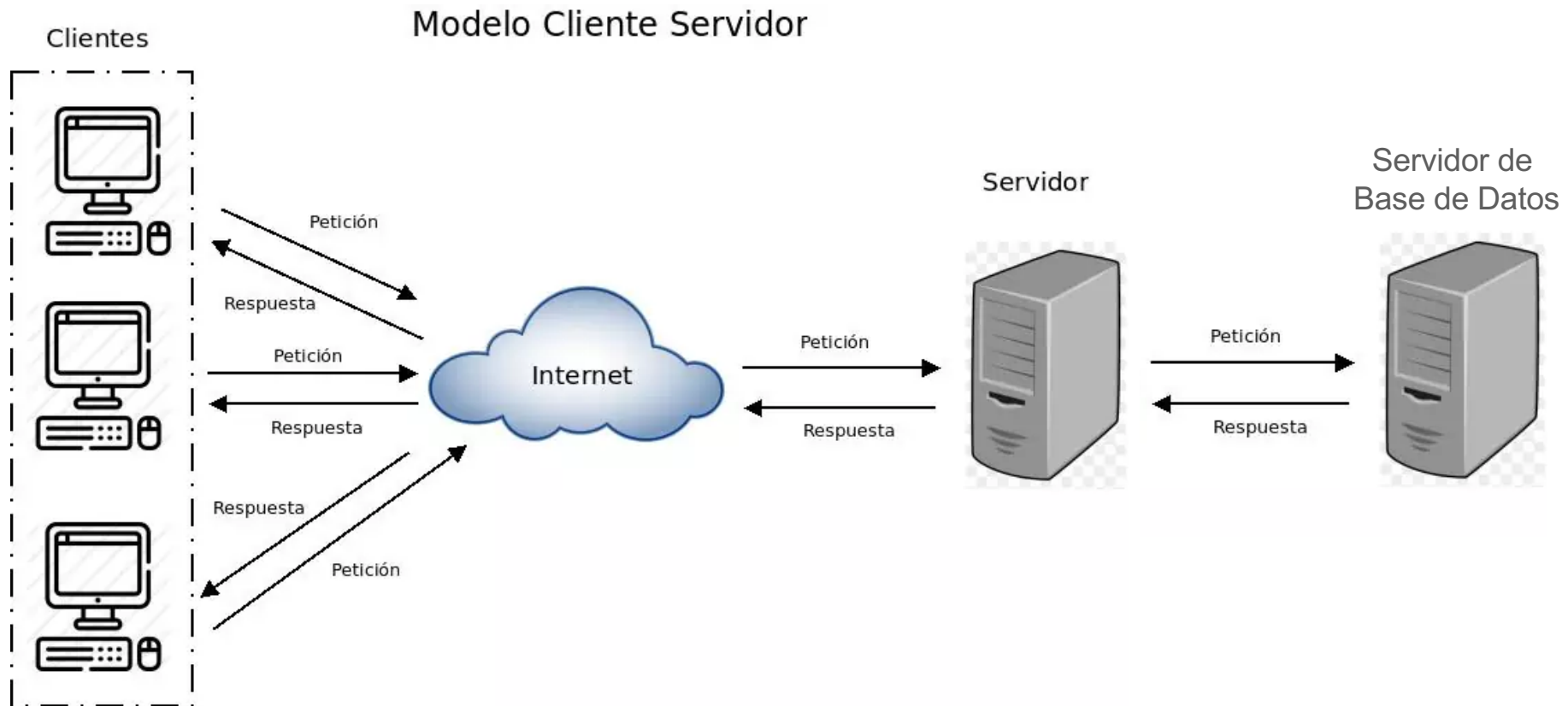
Arquitectura de una aplicación Web

Arquitectura Cliente/Servidor



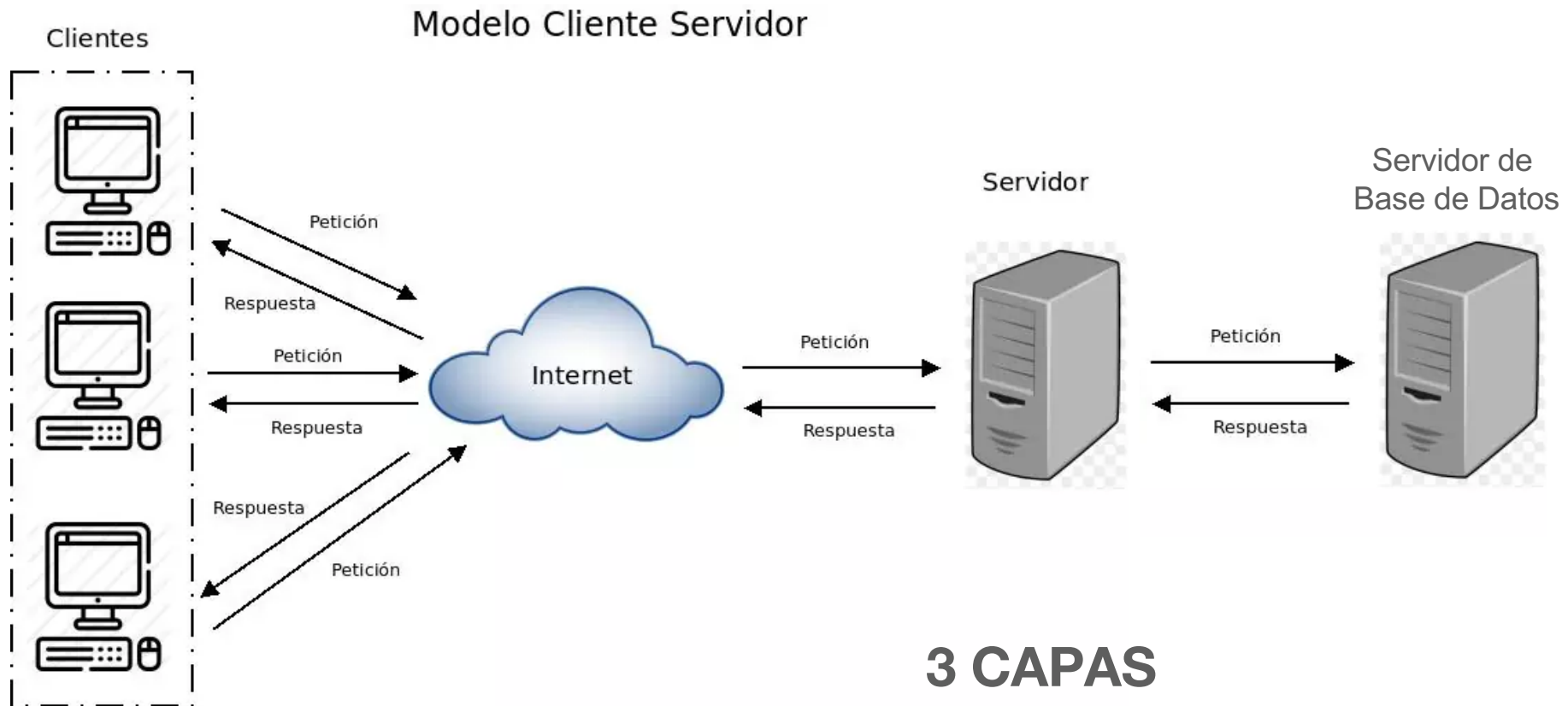
Arquitectura de una aplicación Web

Arquitectura Cliente/Servidor



Arquitectura de una aplicación Web

Arquitectura Cliente/Servidor



¿Cómo se comunican cliente y servidor en internet?

Mediante el modelo de comunicación en capas TCP/IP

Este modelo se basa en **2 protocolos clave** para gestionar toda la comunicación entre equipos en internet.

- **TCP (*Transfer Control Protocol*)**: Se encarga de establecer una conexión segura de intercambio de datos entre dos equipos.

¿Cómo se comunican cliente y servidor en internet?

Mediante el modelo de comunicación en capas **TCP/IP**

Este modelo se basa en **2 protocolos clave** para gestionar toda la comunicación entre equipos en internet.

- **TCP (*Transfer Control Protocol*)**: Se encarga de establecer una conexión segura de intercambio de datos entre dos equipos.
- **IP (*Internet Protocol*)**: Proporciona direcciones numéricas únicas para cada equipo de la red, que pueden ser de distinto tamaño según la versión:

- **IPv4**: Secuencias de 4 números de 8 bits (32 bits en total). Por ejemplo:

192.168.1.1

- **IPv6**: Secuencias de 8 números de 16 bits () representados en hexadecimal y separados por ":". Por ejemplo:

2001:0000:3238:DFE1:63:0000:0000:FEFB.

Servidores y Páginas Web

Los **servidores web** (también conocidos como **servidores HTTP**) son un tipo de servidores utilizados para la distribución (entrega) de contenido web en redes internas o en Internet.

El protocolo HTTP define **un conjunto de métodos de petición** que indica que queremos hacer con un recurso concreto de la página web.

Método	¿Qué hace?	¿Modifica algo?
GET	Pide un recurso (una página, una imagen, etc.) con su información presente en la URL	No
POST	Envía datos al servidor (como un formulario) de manera oculta, sin mostrarlos en la URL	Sí
PUT	Reemplaza un recurso existente con la información proporcionada en la petición	Sí
DELETE	Elimina un recurso específico indicado en la petición	Sí

Servidores y Páginas Web

Servidores Web HW vs Servidores Web SW

Es muy común la **confusión de servidores web** entre las **maquinas hardware que contiene los servicios** y el **software que permite el acceso y gestiona las páginas web**.

Servidores Web HW:

- Son la clasificación de **servidores** que se pueden encontrar por internet a la hora de **contratar un servicio de hosting web**.
- **Limitan los recursos** de hardware que pueden utilizarse **en función de lo contratado**.
- **Simularán una máquina donde podemos instalar el software** necesario para subir y mantener las páginas web.

Servidores y Páginas Web

Servidores Web HW (cont.)

- Podemos encontrar diferentes tipos:
 - **Servidor compartido o Web Hosting:** El hardware es **compartido con otros clientes** por lo que los recursos son limitados y **las ampliaciones exigen pago** de tarifas **extra**.
 - **Servidor dedicado:** Es como tener una **máquina propia**. Permite configuraciones más avanzadas y el **mantenimiento suele recaer en el propio cliente**. Suelen ser **más caros**.
 - **Servidor Virtual Privado (VPS):** Se gestiona como una **máquina virtual** y los **recursos** de los que se dispone **pueden ampliarse automáticamente** bajo demanda reduciéndose cuando la carga decae.
 - **Infraestructura como Servicio (IaaS):** Es un sistema donde **el cliente puede alojar todos sus recursos** (hardware, software, almacenamiento, datos...). Los clientes solo pagan por lo que utilizan y los clientes saben que su aplicación está en un lugar **seguro y escalable**. Ejemplos de este tipo serían **Amazon Web Services** o **Microsoft Azure**.

Servidores y Páginas Web

Servidores Web SW

Servidores Web SW:

- Son el **software** que permite la **gestión web**.
- Los más destacados son:
 - **Apache HTTP:** Es de **código abierto** y **multiplataforma** (sistemas UNIX, Microsoft y MacOS). Es **sencillo**, **ampliable** y recibe actualizaciones y parches con frecuencia.
 - **Internet Information Services (IIS):** Es un servidor pensado para **sistemas Windows**. Es el más utilizado para aplicaciones **ASP.NET** y compatible con la paquetería de Windows. Para montarlo se deberá tener un **hosting Windows**.



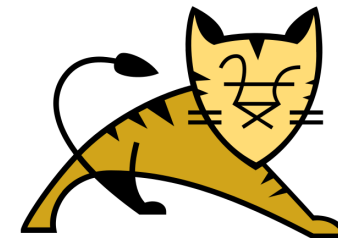
Servidores y Páginas Web

Servidores Web SW

- **NGINX:** Es de **código abierto y multiplataforma**, compatible con sistemas UNIX, Microsoft y MacOS. Destaca por su **alto rendimiento**, ya que puede manejar **más de diez mil conexiones simultaneas** haciendo un gasto de memoria bajo, proporcionando gran escalabilidad a bajo coste.



- **Apache Tomcat:** Es un servidor web y contenedor de servlets que ejecuta **aplicaciones web basadas en Java**. Permite que las app en Java respondan directamente peticiones HTTP. Es de **código abierto y multiplataforma** y funciona siempre que el JDK esté instalado. Puede integrarse con Apache HTTP.



Apache Tomcat

Servidores y Páginas Web

Páginas Web

Las páginas web pueden clasificarse en **dos tipos**:

Páginas web estáticas:

Su información no varía según la hora, lugar de conexión ni ningún elemento externo. Suelen ser **HTML** y tener alguna **funcionalidad en el lado cliente**. Ejemplo: Página que describe los servicios y la información de un negocio, los cuales no varían constantemente.

Páginas web dinámicas:

La página **se genera en el momento de la petición** con una **consulta a la base de datos**. El contenido del HTML y los estilos se generan en ese momento. Ejemplo: **La página web de un periódico**.

Tiendas virtuales, gestores de contenidos o aplicaciones web son las páginas web dinámicas más extendidas.

Lenguajes de Programación

Las páginas web dinámicas se dividen en dos partes:

- **Frontend:** Es la parte que puede ver el usuario final cuando accede a la página. Define donde está colocado y que aspecto tiene cada elemento de la página web.
- **Backend:** Es la parte que se encarga de que toda la lógica de una página web funcione. Definiendo que acción hará cada botón de un modo transparente para el usuario.

Dependiendo de la parte en la que se programen las funcionalidades existen diferentes lenguajes de programación

Lenguajes de Programación

Lado Cliente



- **JavaScript:** Es el **lenguaje más usado** en el lado cliente. Es un lenguaje **interpretado, dinámico, débilmente tipado y multiplataforma** que se ejecuta en los navegadores web.



- **TypeScript:** **Superconjunto tipado de JavaScript.** Es un lenguaje **compilado** e incluye herramientas que permite **autocompletado o refactorización** y soporte para **orientación a objetos**.



- **WebAssembly:** Formato **binario portable** que permite ejecutar código de bajo nivel **C/C++/Rust** con gran rendimiento. Puede superar a JavaScript en algunos escenarios de trabajo intensivo.

Lenguajes de Programación

Lado Cliente



- **Svelte: Compilador de componentes web** que convierte el código en JavaScript óptimo y eficiente.



- **Solid.js: Framework de JavaScript** reactivo y declarativo que destaca por su alta velocidad de renderizado y su tamaño pequeño.



- **Stencil: Compilador para generar Componentes Web Reutilizables** que funcionan en cualquier marco de front-end moderno.



- **Lit: Biblioteca ligera para crear componentes web** rápidos y modernos y **reutilizables** utilizando APIs web estándar como Web Components.

Lenguajes de Programación

Lado Servidor



- **Python:** Es un **lenguaje de propósito general**, fácil de aprender y con una **gran cantidad de bibliotecas** para todo tipo de tareas.



- **PHP:** **Ampliamente utilizado** para el desarrollo web, especialmente en aplicaciones de **pequeña y mediana escala**. Es **fácil de aprender**, tiene un **hosting económico** y cuenta con **múltiples frameworks** como Symfony, Laravel o CodeIgniter.



- **Java:** Lenguaje muy antiguo pero **aun muy utilizado**, especialmente **en aplicaciones empresariales y de gran escala**. Destaca por su **robustez, rendimiento** y una gran gama de **herramientas de desarrollo maduras**.

Lenguajes de Programación

Lado Servidor



- **Rust:** Lenguaje enfocado en seguridad y rendimiento. Se emplea en sistemas críticos.



- **Go:** Lenguaje simple, concurrente y de alto rendimiento. Es muy popular para el desarrollo de servicios web, aplicaciones de red y sistemas distribuidos.



- **Deno:** Entorno de ejecución seguro y moderno para JavaScript y TypeScript, diseñado como una alternativa más segura y eficiente a Node.js.

Lenguajes de Programación

Lado Servidor



- **Elixir:** Lenguaje funcional diseñado para construir aplicaciones escalables y tolerantes a fallos basadas en la máquina virtual Erlang.



- **Kotlin:** Lenguaje moderno y seguro que se utiliza cada vez más en el desarrollo de servidores, especialmente en el ecosistema Java. Combina características de programación orientada a objetos y programación funcional.



- **Swift:** Inicialmente diseñado por Apple para el desarrollo de aplicaciones iOS, también se utiliza para el desarrollo del lado del servidor. Ofrece un rendimiento sobresaliente, seguridad de tipos y una sintaxis moderna.

Preparando el Entorno

Herramientas Básicas

- **Navegadores Web:** Software que funciona como cliente web y permite mostrar las páginas web.



- **Tratamiento de imágenes:** Software que permite modificar las imágenes que se mostraran en la web.



- **Herramientas de diagramas UML:** Software diseñado para planificar los diagramas y las funcionalidades de la web. Estos permiten hacer un diagrama sobre la estructura de la web.



Preparando el Entorno

Entornos de programación en servidor

- **Editores de código:** Son **editores de texto con varias funciones** que facilitan el proceso de escritura de código ayudando a **identificar la sintaxis del lenguaje, cambiando el color de las etiquetas, cerrando etiquetas abiertas**, etc.



Atom, Sublime Text o Visual Studio Code son editores de código muy ligeros para trabajar con cualquier tipo de archivos que incluyen la función de instalar paquetes para programar en distintos lenguajes.

Preparando el Entorno

Entornos de programación en servidor

- **Entornos de desarrollo integrado (IDE):** Son herramienta que **se utilizan para escribir, generar, probar y depurar un programa** todo ello bajo una interfaz de usuario. **Pueden manejar múltiples lenguajes** para diferentes tipos de desarrollos o plataformas. Los principales son:



- **Netbeans:** Principalmente usador para trabajar con Java, puede configurarse para utilizar PHP, HTML, CSS...



- **PHPStorm:** Cuenta con multitud de características para el desarrollo web, proporciona acceso e integración con diferentes sistemas de control de versiones, bases de datos...



- **Eclipse:** permite el desarrollo de una inmensa cantidad de lenguajes de programación. Puede trabajar con distintos frameworks de PHP y JavaScript.

Preparando el Entorno

Herramientas de gestión de bases de datos

Para crear paginas dinámicas **los proyectos suelen contar con una base de datos**, que se gestiona mediante un **SGBD** que **se encarga de que se mantenga la integridad de los datos** y que las diferentes operaciones sobre la base de datos no generen un estado inconsistente de la misma.

Los **principales SGBDs utilizados en aplicaciones web** o móviles son:

- *MySQL y MariaDB*
- *SQL Server*
- *PostgreSQL*
- *MongoDB*
- *SQLite*

Preparando el Entorno

Herramientas de administración de bases de datos

Para gestionar el contenido de las bases de datos de un modo más visual, existe software que permite la administración de una manera más cómoda.

Los **más utilizadas** son:

phpMyAdmin: Administrador de base de datos de código abierto, multilenguaje, construido con PHP, cuya interfaz es mediante el navegador web. Principalmente administra bases de datos MySQL y MariaDB.

pgAdmin: Plataforma de código abierto de administración y desarrollo de bases de datos para PostgreSQL más popular.

DBeaver: Herramienta de código abierto multiplataforma con gran adopción para administrar bases de datos relacionales y no relacionales. Soporta una amplia gama de sistemas de bases de datos, como PostgreSQL, MySQL, SQLite, Oracle, SQL Server, MongoDB...

Preparando el Entorno

Herramientas de administración de bases de datos

Adminer: Administrador de base de datos de código abierto, multilenguaje, sencillo y personalizable con PHP. Soporta múltiples bases de datos relacionales y no relacionales, como MySQL, PostgreSQL, SQLite, MongoDB...

Robo 3T: Administrador de base de datos de MongoDB de código abierto, multiplataforma y ligero.

TablePlus: Multiplataforma y de código abierto, soporta bases de datos relacionales como PostgreSQL, MySQL, SQLite, SQL Server y bases de datos NoSQL como MongoDB.

Beekeeper Studio: Moderno y multiplataforma. Soporta bases de datos relacionales como PostgreSQL, MySQL, SQLite, SQL Server y bases de datos NoSQL como MongoDB.

Preparando el Entorno

Pilas de Software

Una **pila de software** (o *software stack*, en inglés) es un **conjunto de tecnologías que trabajan juntas** para que una aplicación funcione. Cada parte de la pila se encarga de una **capa diferente** del sistema.

Cada capa tiene su función:

- **Sistema operativo** (como Linux o Windows) → La base
- **Servidor SW** (como Apache, Nginx) → Maneja las peticiones
- **Lenguaje backend** (como PHP, Java, Python) → Lógica del programa
- **Base de datos** (como MySQL, MongoDB) → Guarda la información
- **Frontend** (HTML, CSS, JavaScript) → Lo que ve el usuario

Herramientas de Control de Versiones (Git y Github)

¿Qué es el control de versiones?

El control de versiones es un sistema que permite **gestionar los cambios realizados en archivos o proyectos a lo largo del tiempo**. Se usa principalmente en el desarrollo de software, pero también se aplica a documentos, diseños y otros tipos de contenido digital.

¿Para qué sirve el control de versiones?

- **Colaboración:** Permite que varias personas trabajen en los mismos archivos sin pisarse el trabajo.
- **Registrar cambios:** Guarda cada modificación realizada en un archivo, quién la hizo y cuándo.
- **Rastrear errores:** Puedes ver en qué momento se introdujo un error y corregirlo.
- **Recuperar versiones anteriores:** Puedes volver a una versión previa si algo sale mal.
- **Experimentar sin riesgo:** Puedes crear “ramas” (*branches*) para probar ideas sin afectar el trabajo principal.

Herramientas de Control de Versiones (Git y Github)

Mi Proyecto en Git



Cada punto de guardado almacena:

- Información sobre **quién, cuándo y por qué** se hizo ese **cambio**.
- **Qué ficheros** se han modificado junto a las **modificaciones hechas**.
- Un **vínculo con la versión anterior** para mantener el historial.

Herramientas de Control de Versiones (Git y Github)

Diferencia entre Git y Github

Git crea todo este espacio en **mi ordenador**



Github almacena los cambios de todos los usuarios **en la nube**



¿Cómo uso git?

1. Instalar Git en tu ordenador

<https://git-scm.com/downloads>

```
~$ sudo apt-get install git
```

2. A partir de aquí utilizaré la terminal usar git

Los comandos más utilizados son:

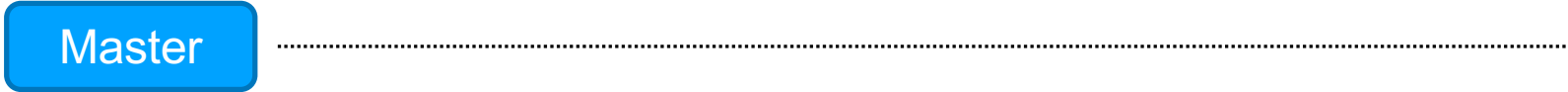
- `git init`
 - `git status`
 - `git add [FILE]`
 - `git commit -m`
 - `git branch`
 - `git branch [NAME]`
 - `git checkout`
 - `git push`
 - `git pull`
- 
- GITHUB**

Integrando git en mi proyecto

1. Elijo la carpeta donde colocare los archivos a controlar e inicializo el repositorio

- **git init:** Sirve para inicializar nuestro repositorio. Crea las medidas necesarias para que se realice el control de versiones.

Master



2. Creo los archivos iniciales que quiero formen parte de mi proyecto

- Ejemplo. Creo un archivo **file.html** con el texto “Hola mundo”

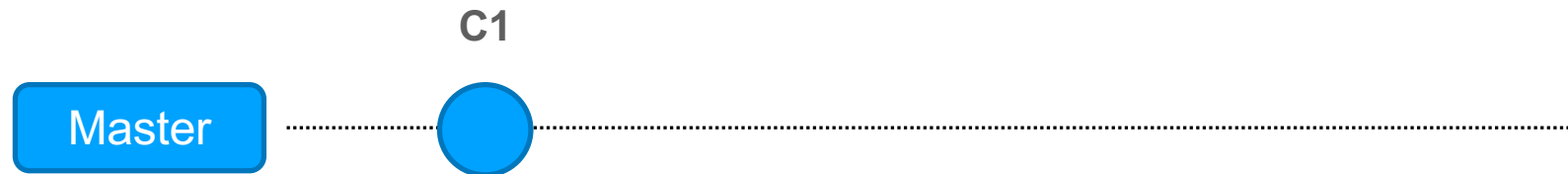
3. Consulto los archivos integrados en el control de versiones y añado nuevos

- **git status:** Me sirve para ver qué archivos son seguidos por *git*
- **git add [Nombre_Archivo]:** Añado los archivos que he creado o modificado y que quiero que se guarden. *Ejemplo: **git add file.html***

Comandos git

4. Creo un punto de guardado (*commit*) de todos los archivos modificados y añadidos con git add

- **git commit -m "Texto_asociado"**: Sirve para **crear un punto de guardado**. Los cambios se guardarán tal cual estén los archivos en este momento. **Texto_asociado** debe tener información de qué cambios he introducido para facilitar el seguimiento del proyecto.

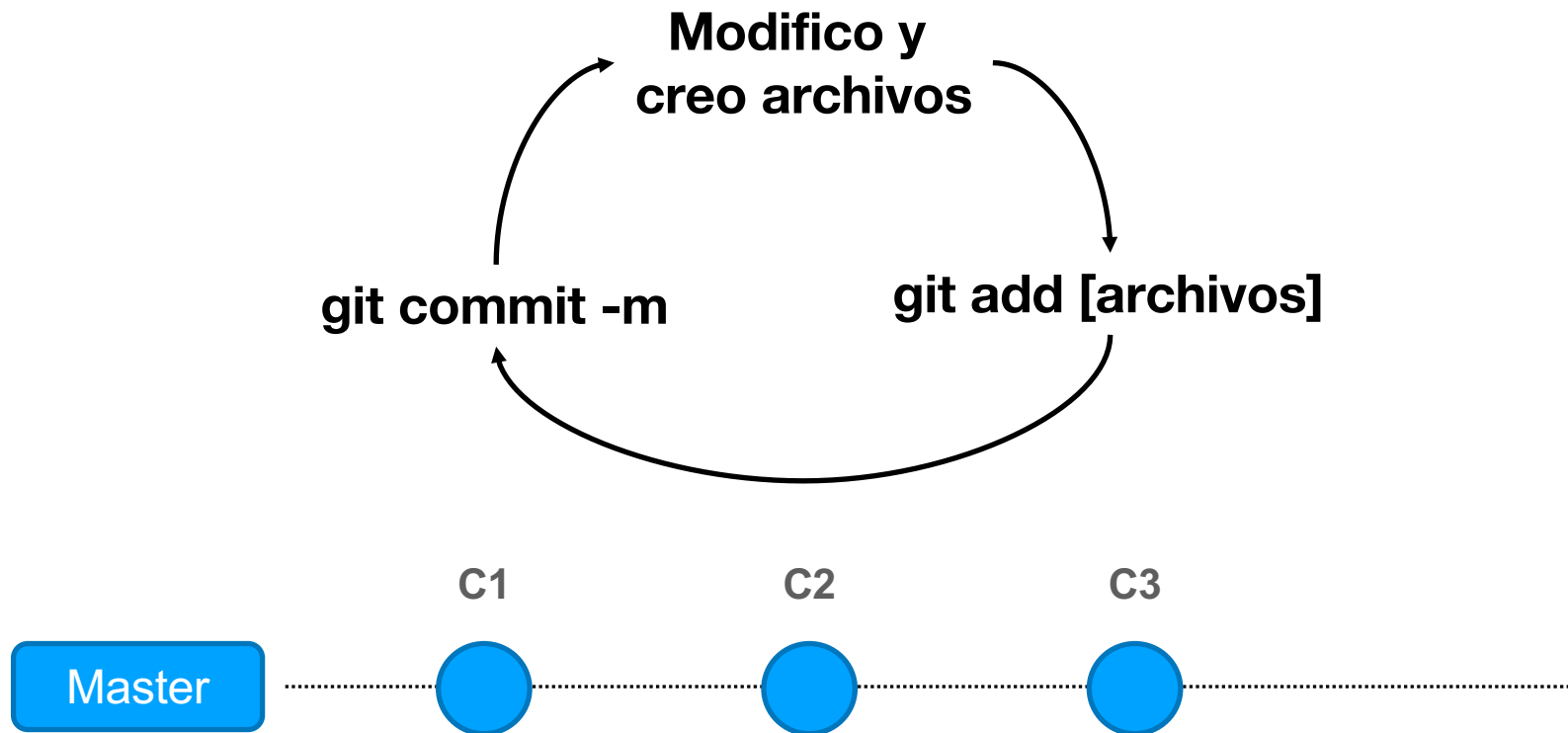


- **git log**: Me sirve para ver todos los commits que ha habido en la rama en la que estoy. Cada commit tiene un código alfanumérico asociado que me permitirá consultar como estaba el proyecto en ese momento.

Ejemplo: **commit 976dbf2ff01a5888422f81e4594ac899be61bb9e**

Comandos git

A partir de aquí entraré en un bucle de trabajo constante:



Repositorios git: RAMAS

Git se utiliza para que varias personas trabajen en paralelo. Será una buena práctica **el desarrollo de nuevas características de manera separada** al proyecto principal, para **evitar que la introducción nuevas característica y cambios** en desarrollo **interfieran con la funcionalidad** principal de la aplicación. Para hacer esto se crean las conocidas como **ramas** (*branches*).

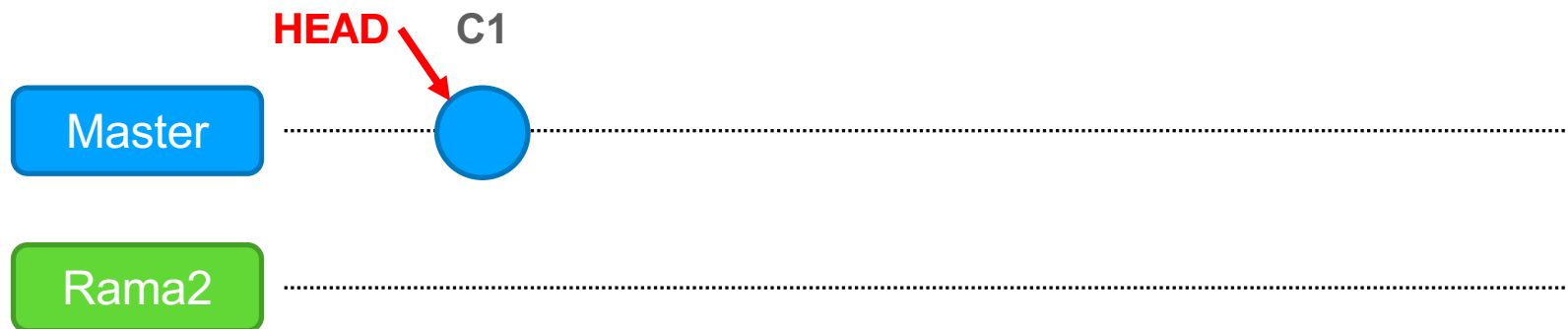
Las ramas se crearán partiendo del commit en el que me encuentro en este momento, de manera que es importante saber **dónde estoy**. Esto puede verse indicado con la palabra **HEAD** cuando ejecutamos el comando **git log**.

Para ver las ramas existentes (o en qué rama estamos) y crear nuevas ramas utilizaremos el comando **git branch**.

- **git branch:** Nos muestra las *branches* que hay en el proyecto, indicándonos con un * la rama en la que estamos ahora mismo.

Repositorios git: RAMAS

- **git branch [NOMBRE]**: Creará una nueva rama con el **NOMBRE** que indiquemos que tendrá una copia del commit en el que estamos. **Ejemplo: git branch Rama2.**



Si quiero cambiarme de branch a **Rama2** deberé utilizar otro comando:

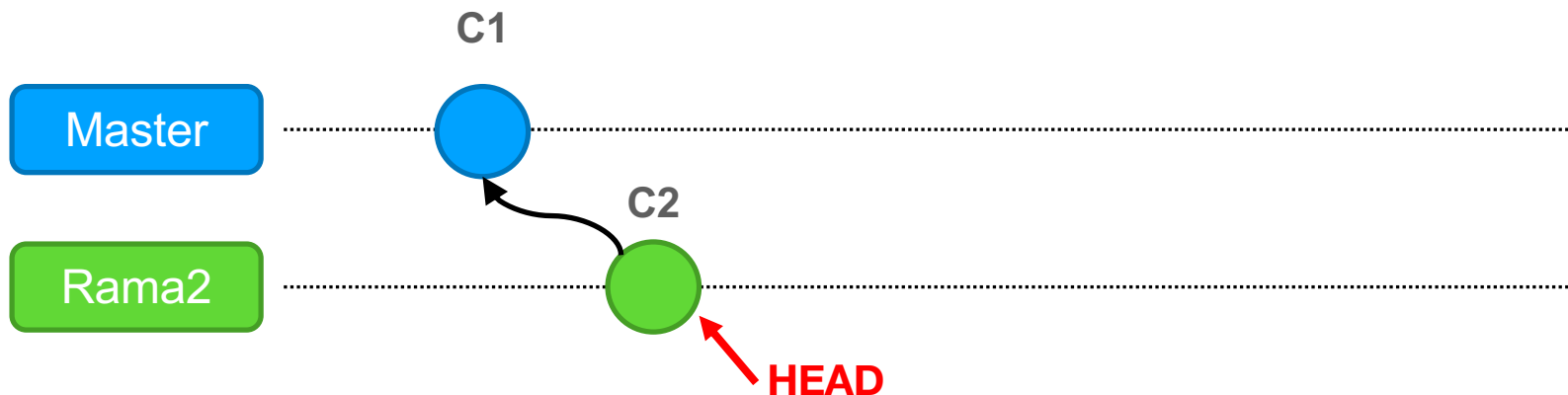
- **git checkout [BRANCH_NAME]**: Este comando cambia mi lugar de trabajo a un entorno nuevo dentro de **Rama2** y las modificaciones que haga ahora no afectarán a los archivos en **Master**. *Este comando también se puede usar para mover el HEAD a commits antiguos utilizando el código del commit.*

Repositorios git: RAMAS

Ejemplo: *git checkout Rama2* moverá el **HEAD** a Rama2



- Al hacer un nuevo commit los archivos evolucionarán dentro de Rama2



Relación entre Git y Github

Todo lo que hemos visto hasta ahora sucede en local

Si vamos a colaborar deberemos tener nuestros repositorios en la nube y sincronizarlos para que todo el mundo acceda.



www.github.com

Debemos **crear una cuenta** y poner nuestras credenciales en nuestro repositorio de git local.

Relación entre Git y Github

Configurar las credenciales en la terminal:

```
git config --global user.name USUARIO_GITHUB
```

```
git config --global user.email EMAIL
```

```
git config -- global user.password TOKEN_AUTENTICACION
```

El token de autenticación es necesario para poder volcar la información de nuestro repositorio local al repositorio en la nube de github utilizando la terminal. Tenéis un vídeo de cómo generar y usar este token en AULES.

Sincronización Git y Github

Por último veremos como sincronizar nuestro trabajo con Github o como descargar a nuestro repositorio local los cambios de otros usuarios con 2 comandos.

1. Creo un nuevo repositorio en Github desde la interfaz

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * **Repository name ***

 serperu-trial / Nuevo_Repo

✓ Nuevo_Repo is available.

Great repository names are short and memorable. How about **solid-guacamole**?

Description

0 / 350 characters

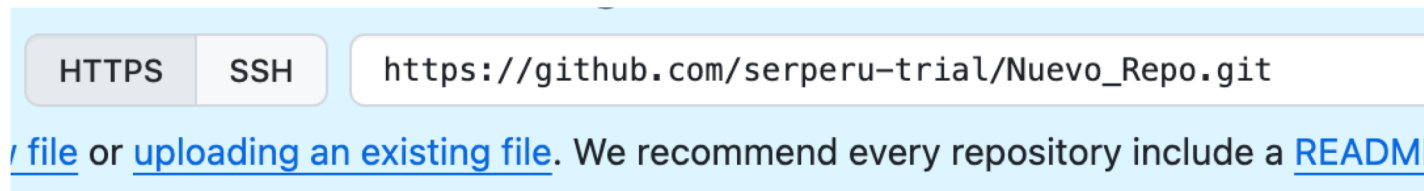
2 Configuration

Choose visibility *
Choose who can see and commit to this repository

 Public

Sincronización Git y Github

2. Vinculo mi repositorio local con el repositorio de Github utilizando la URL que se genera al crearlo.



```
git remote add origin https://github.com/serperu-trial/Nuevo_Repo.git
```

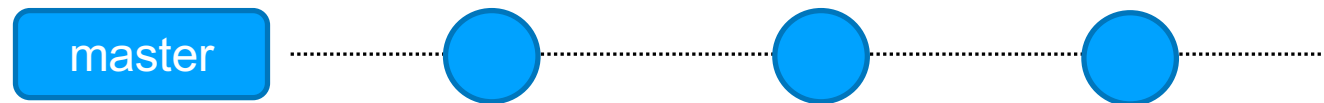
Este comando vinculará ambos repositorios y os pedirá la **contraseña** que **será el token** generado previamente.

Sincronización Git y Github

3. Subir el contenido de local al repositorio remoto

git push: Vuelca el contenido de nuestro repositorio git local a nuestro repositorio vinculado de github en la nube.

LOCAL

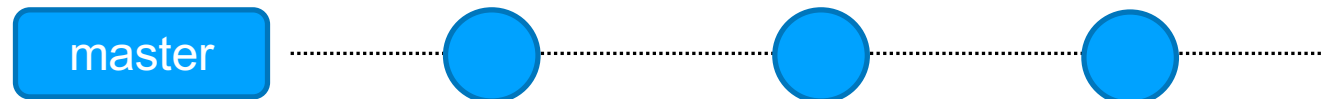


GITHUB



git push -u origin master (*Vincula ambas ramas al sincronizarla a con github. La próxima vez solo tendré que hacer **git push***)

LOCAL



GITHUB

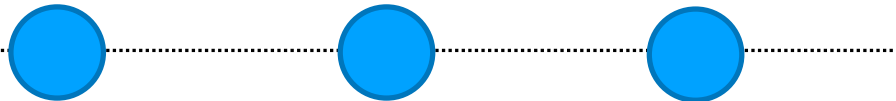


Sincronización Git y Github

Por último puede que la situación sea la inversa y sea **el repositorio Github** el que **tiene la información que queremos sincronizar** para trabajar con ella en local. Para esto utilizaremos el comando **git pull**.

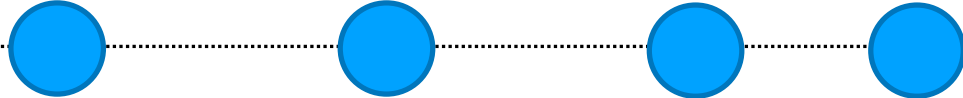
LOCAL

master



GITHUB

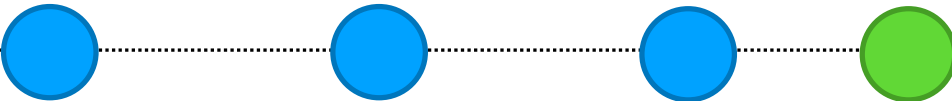
master



git pull origin master (*Esto indica que quiero traer desde el repositorio remoto todo lo que hay en la rama master. Si ejecuto solo **git pull** traeré desde el repositorio remoto los commits de la rama en la que estoy.*)

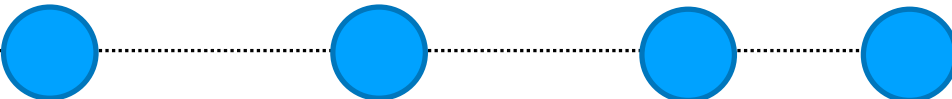
LOCAL

master



GITHUB

master



Sincronización Git y Github

*La comunicación **git-github** puede integrarse fácilmente en diferentes editores de código y entornos de desarrollo como **Visual Studio Code** permitiéndonos sincronizar la información mediante la interfaz del editor*

